

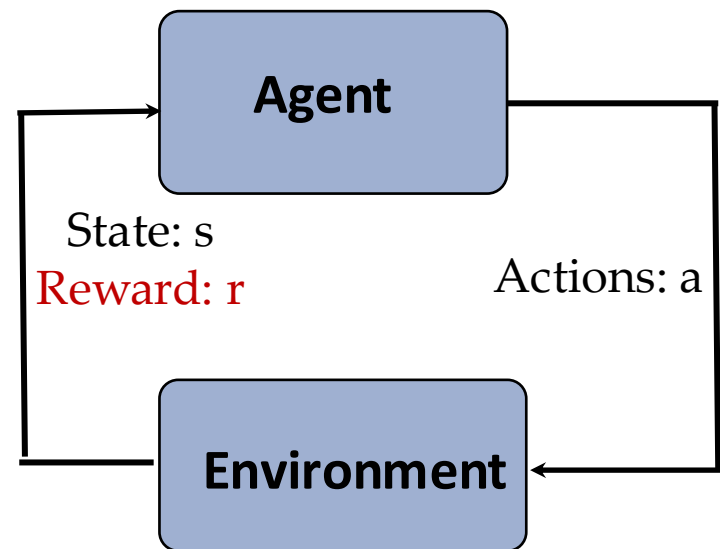
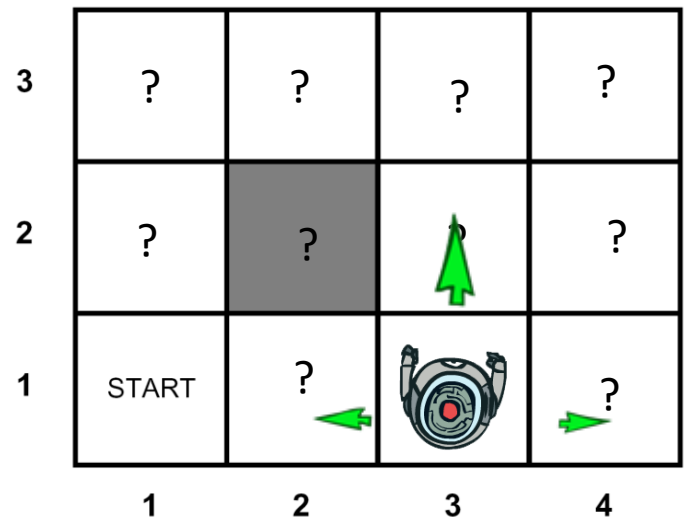
7. Reinforcement Learning I

Ack: Berkeley AI course

Outline

- Reinforcement Learning
 - Model-based RL
 - Model-free RL

Reinforcement Learning



- Basic idea:
 - Receive feedback in the form of **rewards**
 - Agent's utility is defined by the reward function
 - Must (learn to) act so as to **maximize expected rewards**
 - **All learning is based on observed samples of outcomes!**

Example: Learning to Walk



Initial



A Learning Trial



After Learning [1K Trials]

Example: Learning to Walk



Initial

Example: Learning to Walk



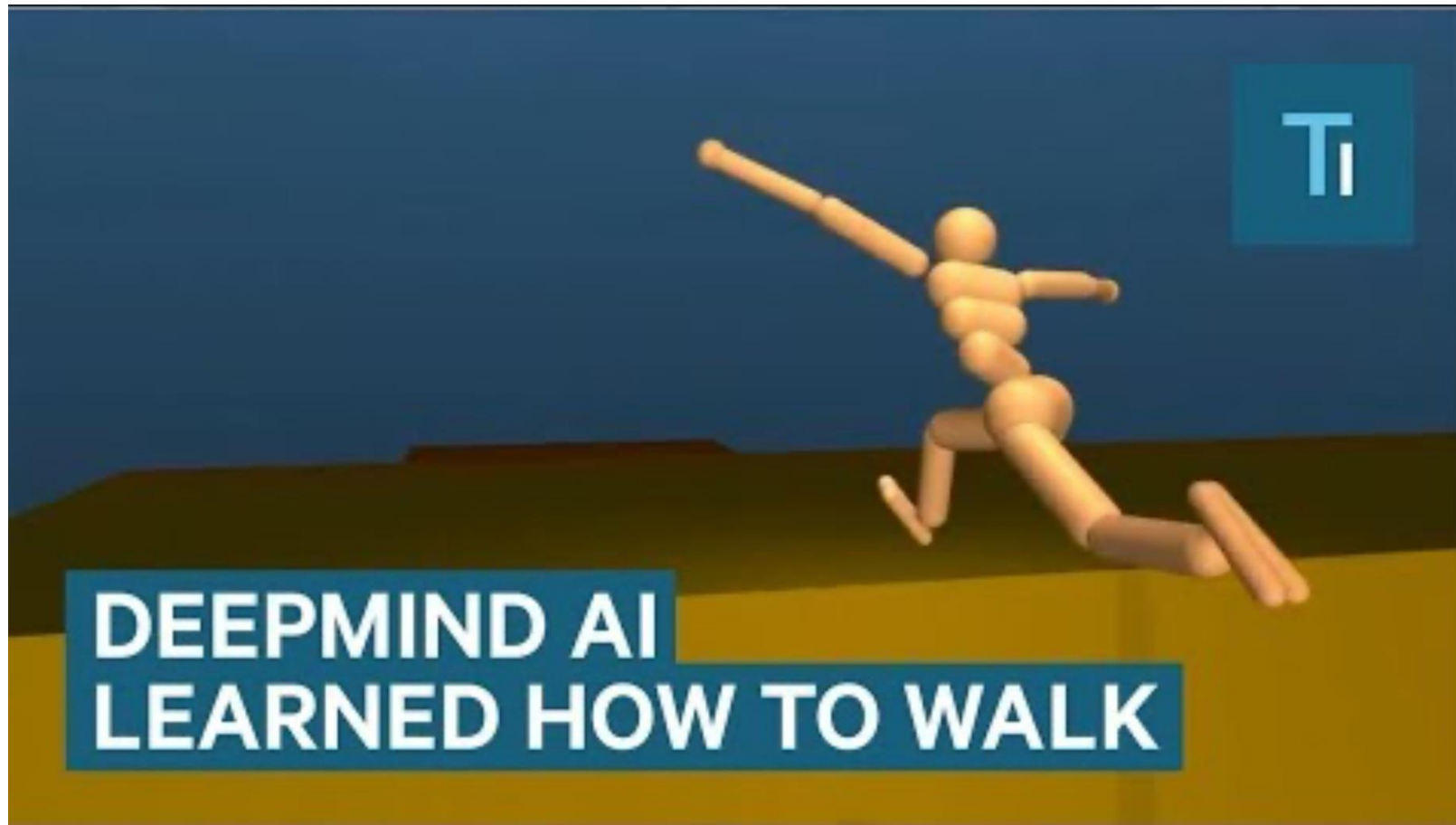
Training

Example: Learning to Walk

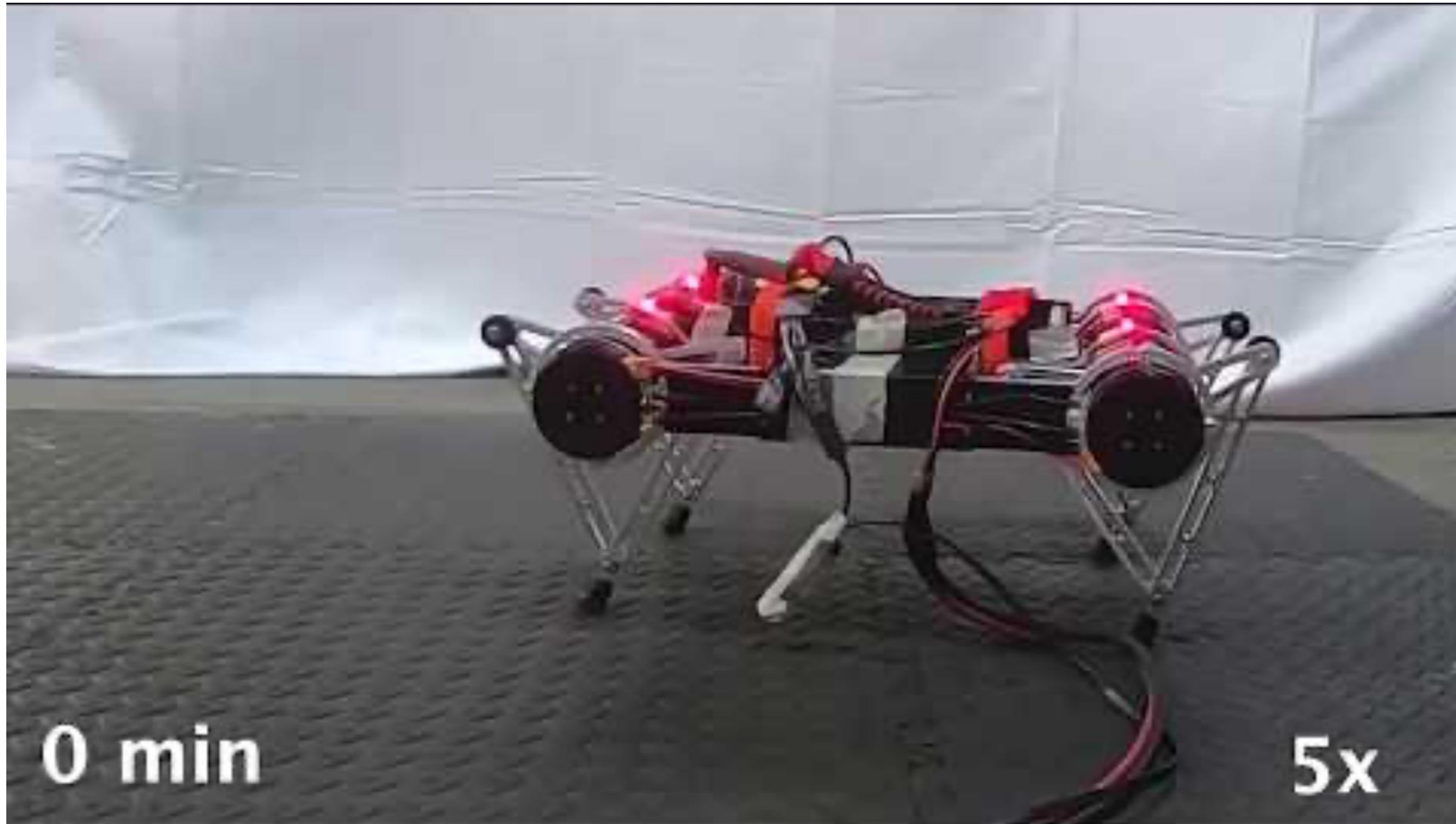


Finished

Demo



Demo



Reinforcement Learning

- Still assume a Markov decision process (MDP):

- A set of states $s \in S$
- A set of actions (per state) A
- A model $T(s,a,s')$
- A reward function $R(s,a,s')$

- Still looking for a policy $\pi(s)$



- New twist: **don't know T or R**

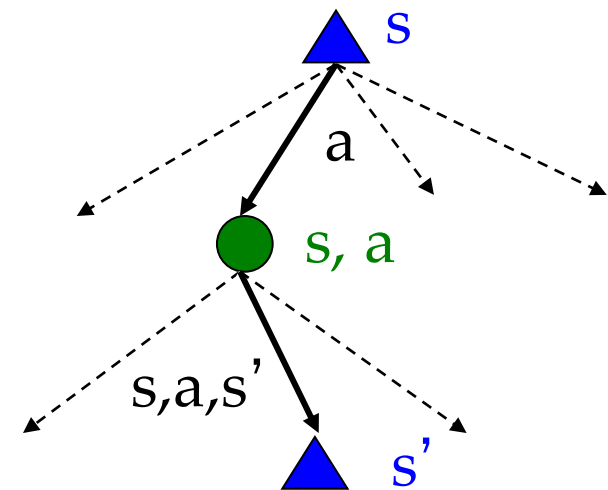
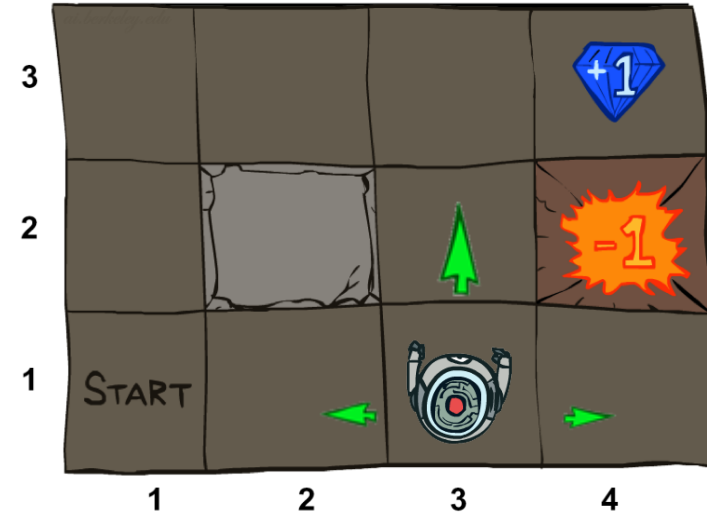
- I.e. we don't know which states are good or what the actions do
- Must actually try actions and states out to learn

trial & learn,

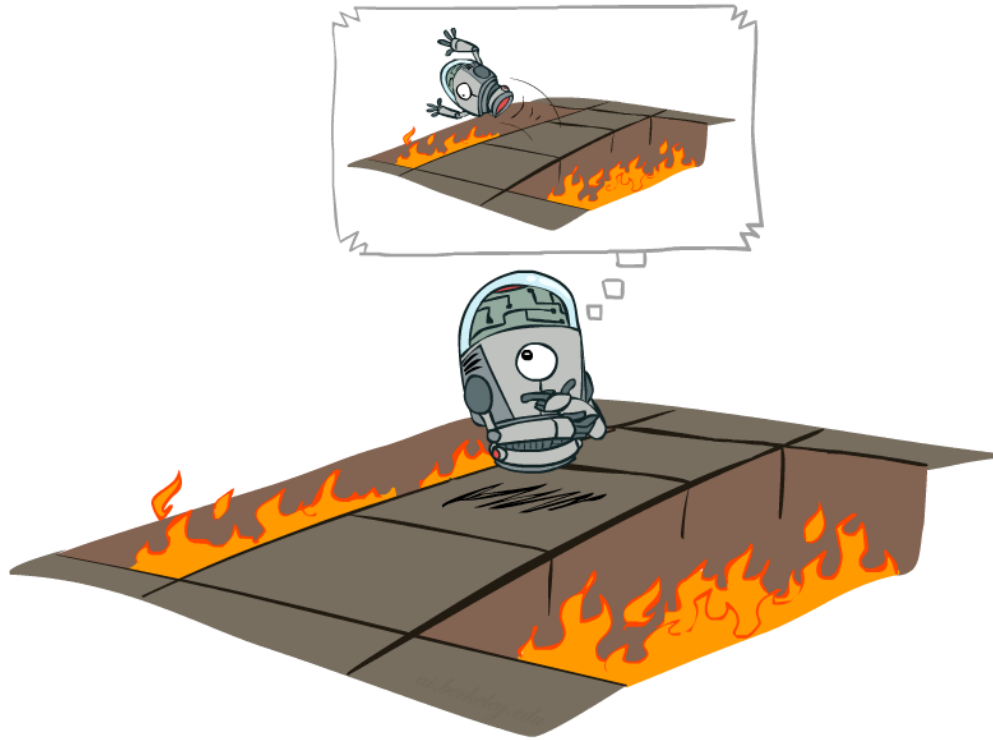
*RL { try: get samples, estimate T or R .
learn: solve mdp*

Recap: MDP

- Markov decision processes:
 - Set of states S
 - Start state s_0
 - Set of actions A
 - Transitions $P(s' | s, a)$ (or $T(s, a, s')$)
 - Rewards $R(s, a, s')$ (and discount γ)
- MDP quantities so far:
 - Policy = Choice of action for each state
 - Utility = sum of (discounted) rewards



Offline (MDPs) vs. Online (RL)



Offline Solution: Imagine what it is to take an action and learn



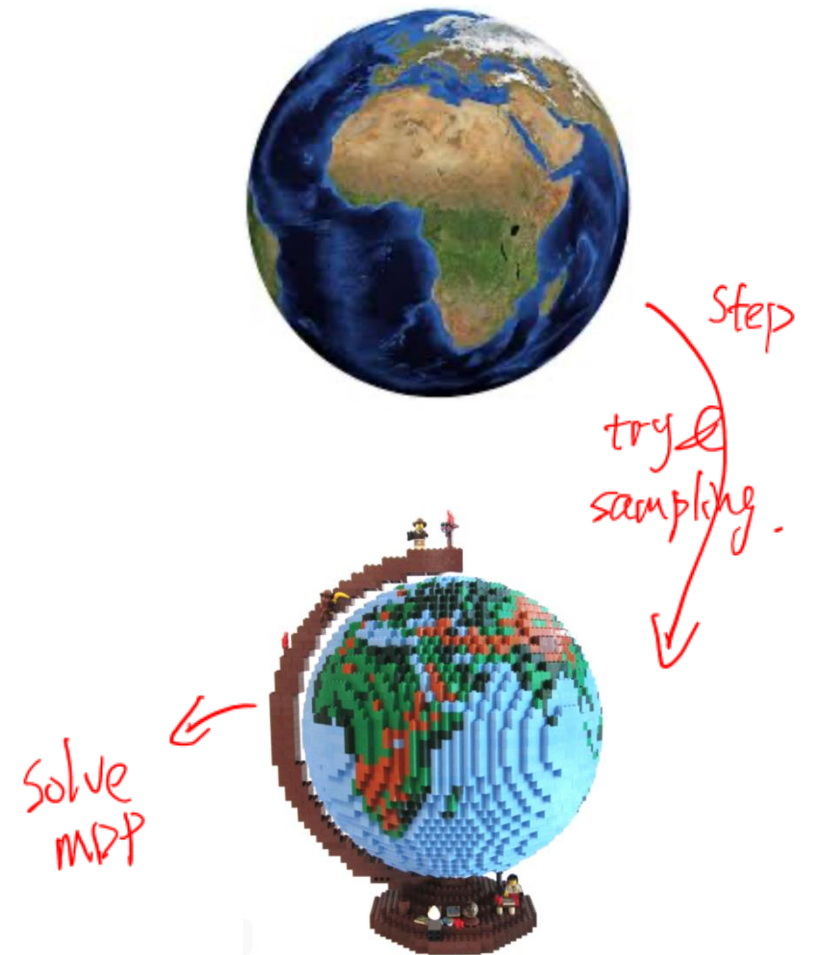
Online Learning: actually take an action and learn

Outline

- Reinforcement Learning
 - **Model-based RL**
 - Model-free RL

Model-Based Reinforcement Learning

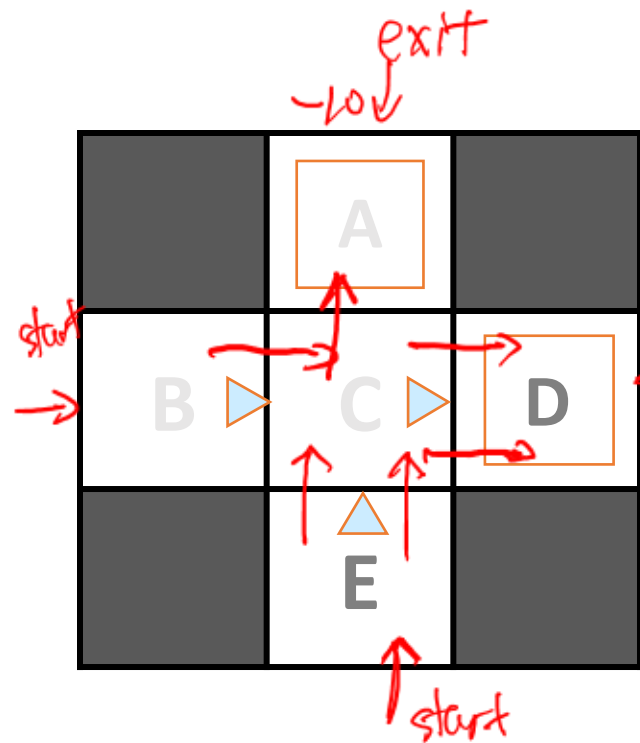
- Model-Based Idea:
 - Learn an approximate model based on experiences
 - Solve for values as if the learned model were correct
- Step 1: Learn empirical MDP model
 - Count outcomes s' for each s, a
 - Normalize to give an estimate of $\hat{T}(s, a, s')$
 - Discover each $\hat{R}(s, a, s')$ when we experience (s, a, s')
- Step 2: Solve the learned MDP
 - For example, use value iteration, as before



Example: Model-Based RL

$$P(D|C, \text{east}) = \frac{3}{4}$$

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1 ✓
C, east, D, -1 ✓
D, exit, x, +10 ✓

Episode 2

B, east, C, -1 ✓
C, east, D, -1 ✓
D, exit, x, +10 ✓

Episode 3

E, north, C, -1 ✓
C, east, D, -1 ✓
D, exit, x, +10 ✓

Episode 4

E, north, C, -1 ✓
C, north, A, -1 ✓
A, exit, x, -10 ✓

Learned Model

$$\hat{T}(s, a, s') = \frac{2}{2} = 1.0$$

$$T(\text{B, east, C}) = 1.00$$

$$T(\text{C, east, D}) = 0.75$$

$$T(\text{C, east, A}) = 0.25$$

...

$$\frac{3}{4}$$

$$\frac{1}{4}$$

$$\hat{R}(s, a, s')$$

$$R(\text{B, east, C}) = -1$$

$$R(\text{C, east, D}) = -1$$

$$R(\text{D, exit, x}) = +10$$

...

Samples $\rightarrow$ model (T, R) $\rightarrow$ Value or policy (solving MDP).

Example: Expected Age

Goal: Compute expected age of CSE 440 students

Known $P(A)$

$$E[A] = \sum_a P(a) \cdot a = 0.35 \times 20 + \dots$$

Without $P(A)$, instead collect samples $[a_1, a_2, \dots, a_N]$

Unknown $P(A)$: "Model Based"

model

$$\hat{P}(a) = \frac{\text{num}(a)}{N}$$

*1 - 20
2 - 21
10 - 19.*

$$E[A] \approx \sum_a \hat{P}(a) \cdot a$$

*$\frac{1}{10} \times 20$
 $+ \frac{2}{10} \times 21 + \frac{3}{10} \times 19.$*

Why does this work? Because eventually you learn the right model.

Unknown $P(A)$: "Model Free"

$$E[A] \approx \frac{1}{N} \sum_i a_i$$

$\frac{1}{N} (19 + 20 + 21 + \dots)$

Why does this work? Because samples appear with the right frequencies.

Outline

- Reinforcement Learning
 - Model-based RL
 - **Model-free RL**

Passive Model-Free Reinforcement Learning

- Simplified task: policy evaluation
 - Input: a fixed policy $\pi(s)$
 - You don't know the transitions $T(s,a,s')$
 - You don't know the rewards $R(s,a,s')$
 - Goal: learn the state values $V(s)$
- In this case:
 - Learner is “along for the ride”
 - No choice about what actions to take
 - Just execute the policy and learn from experience
 - This is NOT offline planning! You actually take actions in the world.

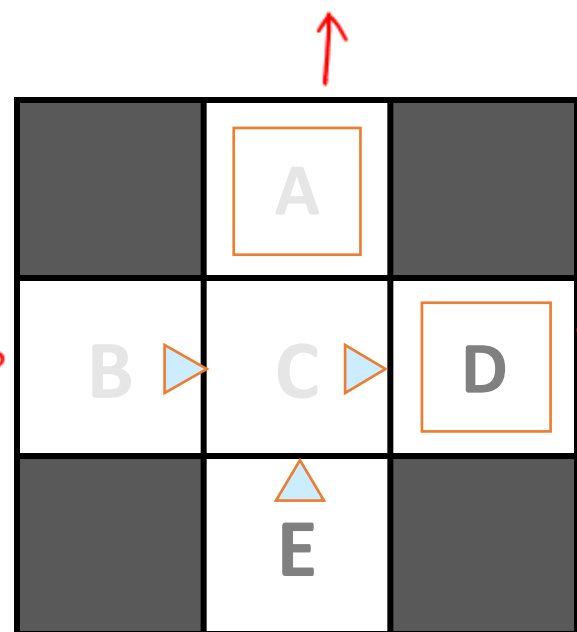
Direct Evaluation

- Goal: Compute values for each state under π
- Idea: Average together observed sample values
 - Act according to π
 - Every time you visit a state, write down what the sum of discounted rewards turned out to be
 - Average those samples
- This is called direct evaluation

1. π fixed.
2. visit states.
compute values.
3. update policy. ✓

Example: Direct Evaluation

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

$$V(B) = \frac{((-1 - 1 + 10) + (-1 - 1 + 10))}{2}$$

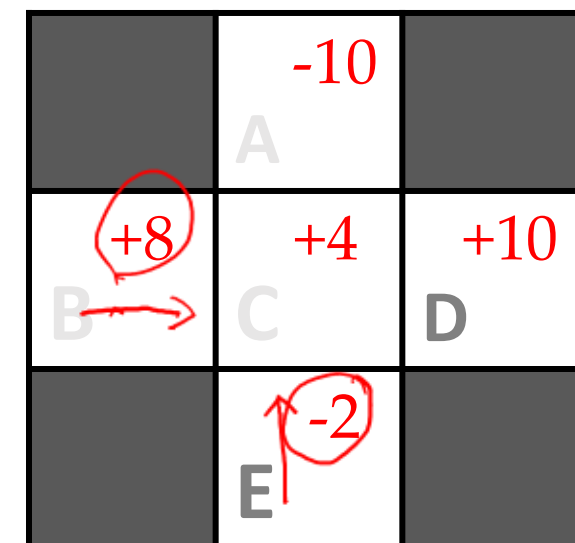
$$= 8$$

$$V(E) = \frac{8 - 12}{2}$$

$$V(C) = \frac{9 + 9 + 9 - 19}{4} = 4$$

$$V(A) = -10$$

$$V(D) = \frac{10 + 10 + 10}{3}$$



If B and E both go to C under this policy, how can their values be different?

$$V^{\pi}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi}(s')]$$

Problems with Direct Evaluation

- What's good about direct evaluation?
 - It's easy to understand
 - It doesn't require any knowledge of T, R
 - It eventually computes the correct average values, using just sample transitions
- What bad about it?
 - It wastes information about state connections
 - Each state must be learned separately
 - So, it takes a long time to learn

Output Values

	-10 A	
+8 B	+4 C	+10 D
	-2 E	

$T(B, \text{east}, C)$

$T(E, \text{north}, C)$

~~E east or south,
west~~

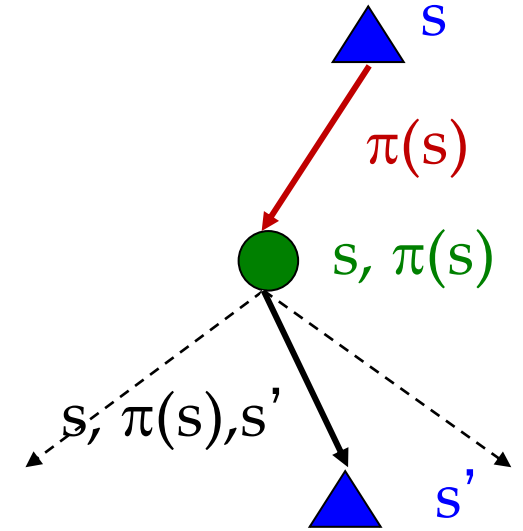
Why Not Use Policy Evaluation?

- Simplified Bellman updates calculate V for a fixed policy:
 - Each round, replace V with a one-step-look-ahead layer over V

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- This approach fully exploited the connections between the states
 - Unfortunately, we need T and R to do it!
- Key question: how can we do this update to V without knowing T and R ?
 - In other words, how to take a weighted average without knowing the weights?



Sample-Based Policy Evaluation?

- We want to improve our estimate of V by computing these averages:

$$V_{k+1}^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^{\pi}(s')]$$

- Idea: Take samples of outcomes s' (by doing the action!) and average

$$\rightarrow sample_1 = \underbrace{R(s, \pi(s), s'_1)}_{\text{TI}} + \underbrace{\gamma V_k^{\pi}(s'_1)}_{\text{TI}}$$

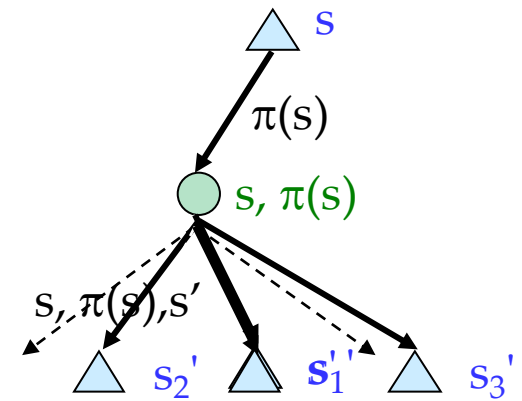
$$sample_2 = R(s, \pi(s), s'_2) + \gamma V_k^{\pi}(s'_2)$$

...

$$sample_n = R(s, \pi(s), s'_n) + \gamma V_k^{\pi}(s'_n)$$

$$\underline{V_{k+1}^{\pi}(s) \leftarrow \frac{1}{n} \sum_i sample_i}$$

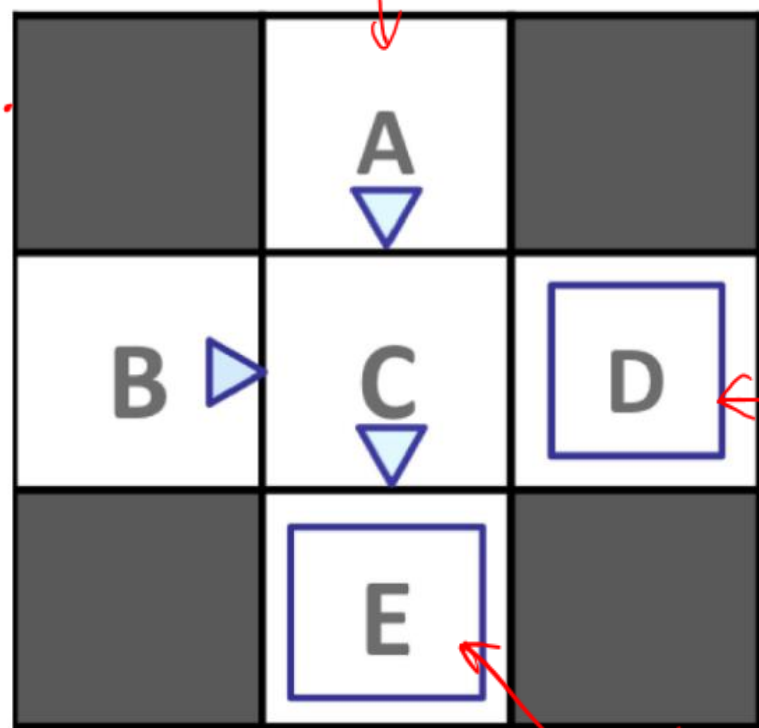
(model-free)



*Almost! But we can't
rewind time to get sample
after sample from state s .*

Input Policy π

Observed Episodes (Training)



Episode 1

A, south, C, -1
C, south, E, -1
E, exit, x, +10

Episode 2

B, east, C, -1
C, south, D, -1
D, exit, x, -10

Episode 3

B, east, C, -1
C, south, E, -1
E, exit, x, +10

Episode 4

A, south, C, -1
C, south, E, -1
E, exit, x, +10

$V(A)$

$V(B)$

$V(C)$

$V(D)$

Assume: $\gamma = 1$

$V(E)$, exit

$\frac{\delta + \gamma V}{1 - \gamma}$